

Raport z walidacji PoC RoguePlanet dla Microsoft Defender

Cel i zakres raportu

Raport dotyczy weryfikacji publicznie opisanego PoC RoguePlanet odnoszącego się do Microsoft Defender. Opisująca technika została przedstawiona w mediach 10 czerwca 2026 r. jako lokalne podniesienie uprawnień (Local Privilege Escalation, LPE), w którym lokalny użytkownik może uzyskać uprawnienia NT AUTHORITY\SYSTEM. W publicznych opisach wskazywano, że mechanizm wykorzystuje funkcje używane przez Microsoft Defender podczas obsługi lub skanowania pliku.

Celem testu było sprawdzenie, czy exploit można przygotować i uruchomić w kontrolowanym środowisku laboratoryjnym, a także jak zachowują się mechanizmy ochronne Microsoft Defender na aktualnym systemie Windows 11. Raport obejmuje opis środowiska testowego, stan aktualizacji, konfigurację Microsoft Defender, przebieg przygotowania środowiska kompilacyjnego, wynik kompilacji, reakcję Defendera oraz rekomendacje ograniczające ryzyko.

Test miał charakter badawczy i został wykonany lokalnie na dedykowanej stacji testowej. Wyniki należy interpretować jako ocenę zachowania konkretnego artefaktu i konkretnej konfiguracji środowiska, a nie jako pełne potwierdzenie odporności na wszystkie możliwe warianty tej techniki.

Źródła wskazane w analizowanym materiale:

- Artykuł:
<https://thehackernews.com/2026/06/microsoft-defender-rogueplanet-zero-day.html>
- Publiczne repozytorium PoC:
<https://github.com/MSNightmare/RoguePlanet/tree/main>
- Źródło instalatora MSYS2:
https://github.com/msys2/msys2-installer/releases/tag/nightly-x86_64
- Źródło Visual Studio:
<https://visualstudio.microsoft.com/insiders/?rwnlp=pl>

Środowisko testowe

PoC przeprowadzono na stacji roboczej typu klient, pracującej poza domeną Active Directory, w grupie roboczej WORKGROUP. System operacyjny zainstalowany na stacji to Microsoft Windows 11 Home, wersja 25H2, architektura 64-bitowa.

Parametr	Wartość
Nazwa systemu	Microsoft Windows 11 Home
Edycja	Home
Wersja systemu	25H2
Wersja OS	10.0.26200
Numer kompilacji	26200
Architektura	x64 / 64-bit
Typ instalacji	Client / Workstation
Nazwa hosta	LAPTOP-80LPIEH2
Producent urządzenia	Lenovo
Model urządzenia	Lenovo Legion Slim 5 16IRH8
Procesor	12th Gen Intel(R) Core(TM) i5-12450H
Pamięć RAM	32 GB

Na dzień wykonania PoC system posiadał zainstalowane aktualizacje bezpieczeństwa z czerwca 2026 r. oraz wcześniejsze aktualizacje z maja i kwietnia 2026 r. Oznacza to, że test został przeprowadzony na aktualnym systemie Windows 11 25H2, build 26200, po instalacji najnowszych dostępnych poprawek bezpieczeństwa na dzień przeprowadzenia testu.

HotFixID	Typ aktualizacji	Data instalacji
KB5094135	Security Update	10.06.2026
KB5094126	Security Update	10.06.2026
KB5087051	Update	14.05.2026
KB5092762	Security Update	13.05.2026
KB5054156	Update	28.04.2026

Konfiguracja Microsoft Defender

Na stacji wykorzystanej do testu aktywny był Microsoft Defender Antivirus działający w trybie normalnym. Usługa ochrony była uruchomiona i włączona, a ochrona antywirusowa, antyspyware, monitor zachowania oraz ochrona w czasie rzeczywistym były aktywne.

Parametr	Wartość
AMProductVersion	4.18.26050.15
AMServiceVersion	4.18.26050.15
AMEngineVersion	1.1.26050.11
AMRunningMode	Normal
AMServiceEnabled	True
AntivirusEnabled	True
AntispywareEnabled	True
RealTimeProtectionEnabled	True
BehaviorMonitorEnabled	True
OnAccessProtectionEnabled	True
IoavProtectionEnabled	True

NISEnabled	True
NISEngineVersion	1.1.26050.11
IsTamperProtected	True
DefenderSignaturesOutOfDate	False
RebootRequired	False
IsVirtualMachine	False

W dniu przeprowadzenia testu sygnatury Microsoft Defender były aktualne. Zarówno sygnatury antywirusowe, antyspyware, jak i NIS zostały zaktualizowane 10.06.2026 o godzinie 13:27:32.

Typ sygnatur	Wersja	Data ostatniej aktualizacji
AntivirusSignatureVersion	1.453.27.0	10.06.2026 13:27:32
AntispywareSignatureVersion	1.453.27.0	10.06.2026 13:27:32
NISSignatureVersion	1.453.27.0	10.06.2026 13:27:32

Ostatnie szybkie skanowanie zostało wykonane 08.06.2026 w godzinach 15:00:36-15:01:58, z wykorzystaniem sygnatur w wersji 1.451.323.0. Pełne skanowanie nie było wcześniej wykonane lub jego historia nie była dostępna, na co wskazuje wartość FullScanAge: 4294967295 oraz brak czasu rozpoczęcia i zakończenia pełnego skanu.

Przygotowanie środowiska kompilacyjnego

Pierwsza próba kompilacji kodu z repozytorium GitHub zakończyła się błędem wynikającym z braku nagłówka winternl.h. Komunikat wskazywał, że w systemie nie było dostępnego kompletnego zestawu nagłówków Windows SDK wymaganego przez analizowany kod.

```
PS C:\mingw\bin> .\g++.exe C:\Users\User\Downloads\RoguePlanet.cpp -o file.exe
C:\Users\User\Downloads\RoguePlanet.cpp:8:22: fatal error: winternl.h: No such file or directory
#include <winternl.h>
```

Rysunek 1. Błąd braku biblioteki winternl.h podczas pierwszej próby kompilacji.

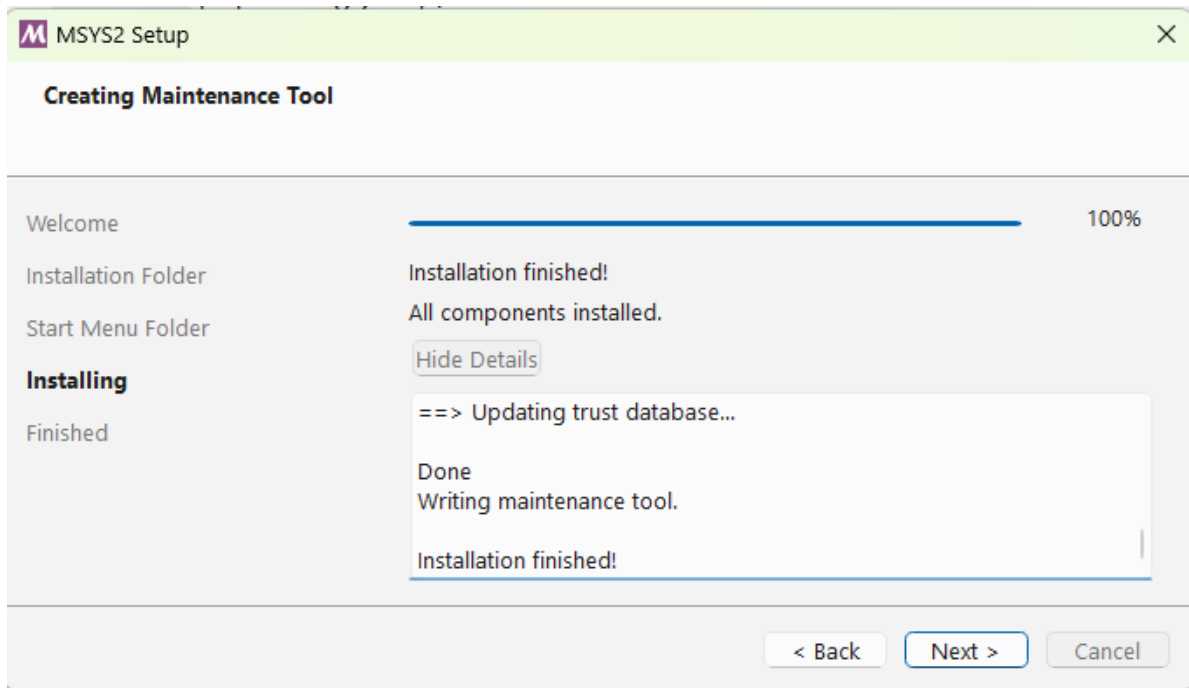
Kod odwoływał się również do innych nagłówków związanych z Windows API oraz NT API, m.in. windows.h, Psapi.h, ntstatus.h, virdisk.h, shlwapi.h, taskschd.h i bcrypt.h. Z tego powodu konieczne było przygotowanie pełniejszego środowiska kompilacyjnego oraz instalacja odpowiednich komponentów SDK.

```
#include <iostream>
#include <Windows.h>
#include <Psapi.h>
#include <winternl.h>
#include <conio.h>
#include <ntstatus.h>
#include <virdisk.h>
#include <shlwapi.h>
#include <initguid.h>
#include <ole2.h>
#include <comdef.h>
#include <taskschd.h>
#include <bcrypt.h>
```

Rysunek 2. Fragment listy nagłówków wymaganych przez analizowany kod.

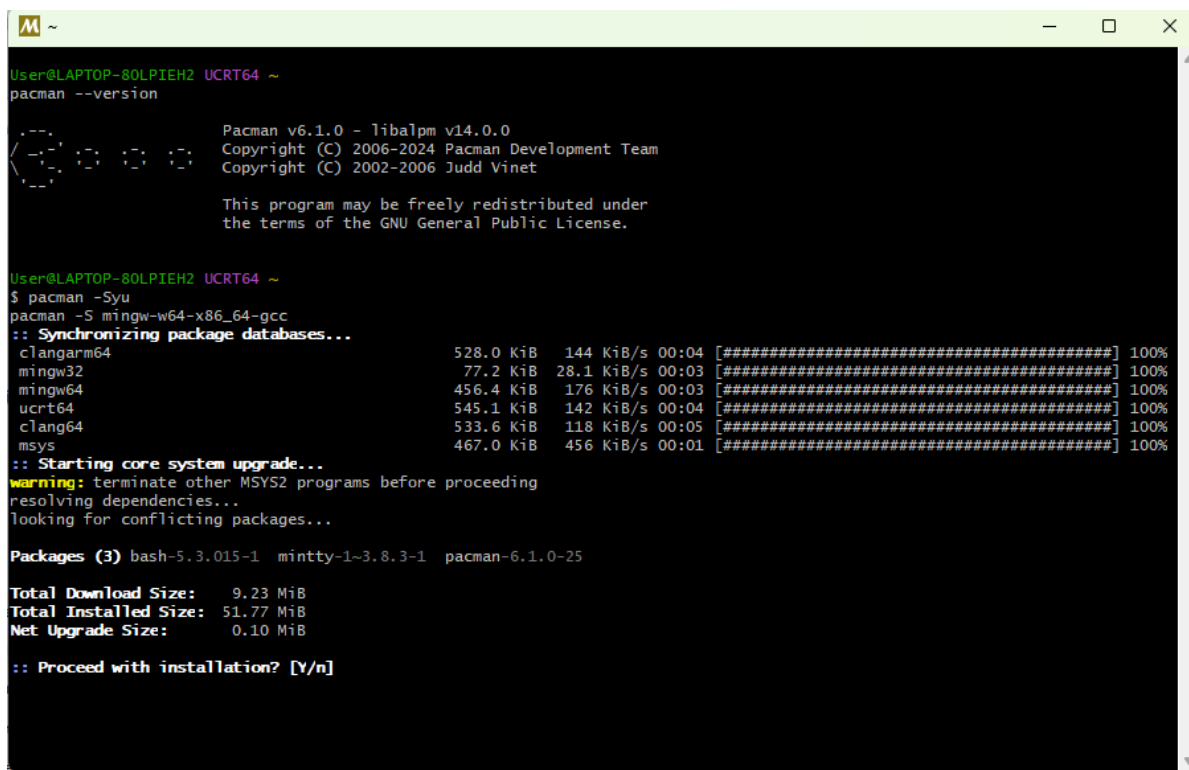
Próba użycia MSYS2/MinGW-w64

Początkowo do przygotowania środowiska kompilacyjnego wykorzystano MSYS2/MinGW-w64. Jest to środowisko udostępniające narzędzia GNU dla systemu Windows, w tym kompilatory gcc i g++. Pakiety w MSYS2 są zarządzane za pomocą narzędzia pacman, które pełni podobną rolę jak apt w systemach Linux lub winget w systemie Windows.



Rysunek 3. Zakończenie instalacji MSYS2.

Za pomocą pacman zainstalowano toolchain MinGW-w64 GCC/G++, czyli zestaw narzędzi umożliwiający kompilację kodu C/C++ dla systemu Windows. Pakiet obejmuje m.in. kompilator gcc, kompilator C++ g++, linker oraz nagłówki i biblioteki wymagane do budowania aplikacji działających w środowisku Windows. Celem tej próby było sprawdzenie, czy kod da się skompilować za pomocą otwartego toolchaina dostępnego w MSYS2, bez użycia środowiska Visual Studio.



```
User@LAPTOP-80LPIEH2 UCRT64 ~
pacman --version

Pacman v6.1.0 - libalpm v14.0.0
Copyright (C) 2006-2024 Pacman Development Team
Copyright (C) 2002-2006 Judd Vinet

This program may be freely redistributed under
the terms of the GNU General Public License.

User@LAPTOP-80LPIEH2 UCRT64 ~
$ pacman -Syu
pacman -S mingw-w64-x86_64-gcc
:: Synchronizing package databases...
clangarm64                    528.0 KiB   144 KiB/s   00:04 [#####] 100%
mingw32                       77.2 KiB   28.1 KiB/s   00:03 [#####] 100%
mingw64                      456.4 KiB   176 KiB/s   00:03 [#####] 100%
ucrt64                       545.1 KiB   142 KiB/s   00:04 [#####] 100%
clang64                      533.6 KiB   118 KiB/s   00:05 [#####] 100%
msys                         467.0 KiB    45 KiB/s   00:01 [#####] 100%
:: Starting core system upgrade...
warning: terminate other MSYS2 programs before proceeding
resolving dependencies...
looking for conflicting packages...

Packages (3) bash-5.3.015-1  mintty-1~3.8.3-1  pacman-6.1.0-25

Total Download Size:    9.23 MiB
Total Installed Size:  51.77 MiB
Net Upgrade Size:       0.10 MiB

:: Proceed with installation? [Y/n]
```

Rysunek 4. Instalacja pakietów MSYS2/MinGW-w64 z użyciem pacman.

Po instalacji wykonano próbę kompilacji kodu za pomocą narzędzia g++. Użyta komenda wskazywała bezpośrednio ścieżkę do pliku źródłowego oraz ścieżkę wynikowego pliku wykonywalnego.

```
C:\msys64\mingw64\bin\g++.exe C:\Users\User\Downloads\RoguePlanet.cpp -o
C:\Users\User\Desktop\roguePlanet.exe
```

Problem trybu Unicode

Pierwszą istotną przesłanką wskazującą na problem z trybem Unicode były komunikaty kompilatora dotyczące niezgodności typów znakowych. W logach pojawiały się błędy informujące, że nie można przekonwertować wartości typu `const wchar_t*` albo `wchar_t*` na typy `LPCSTR` lub `LPSTR`. Oznaczało to, że kod przekazywał do funkcji Windows API napisy szerokoznakowe, natomiast kompilator wybierał warianty funkcji przeznaczone dla klasycznych napisów ANSI.

W Windows API wiele funkcji występuje w dwóch wariantach: ANSI, oznaczonym sufiksem A, oraz Unicode, oznaczonym sufiksem W. Przykładowo `CreateFile` może zostać zmapowane jako `CreateFileA` albo `CreateFileW`, a `RegOpenKeyEx` jako `RegOpenKeyExA` albo `RegOpenKeyExW`. Wariant A oczekuje parametrów typu `char*` lub `LPCSTR`, natomiast wariant W oczekuje parametrów typu `wchar_t*` lub `LPCWSTR`.

W analizowanym przypadku kod używał literatów w postaci L"..." oraz buforów typu wchar_t. Jednocześnie komunikaty błędów wskazywały, że kompilator wybierał funkcje takie jak GetModuleHandleA, RegOpenKeyExA, RegQueryValueExA, GetWindowsDirectoryA, CreateFileA czy wsprintfA. Był to bezpośredni sygnał, że kod jest pisany z założeniem trybu Unicode, ale komenda kompilacji nie miała ustawionych definicji UNICODE oraz _UNICODE.

Wymuszono zatem tryb Unicode poprzez dodanie definicji UNICODE oraz _UNICODE. Po tej zmianie funkcje Windows API bez jawnego sufiksu powinny zostać zmapowane na warianty z sufiksem W, np. CreateFileW, RegOpenKeyExW, GetModuleHandleW i GetWindowsDirectoryW. Fakt, że po tej zmianie część błędów zniknęła, potwierdził poprawność diagnozy.

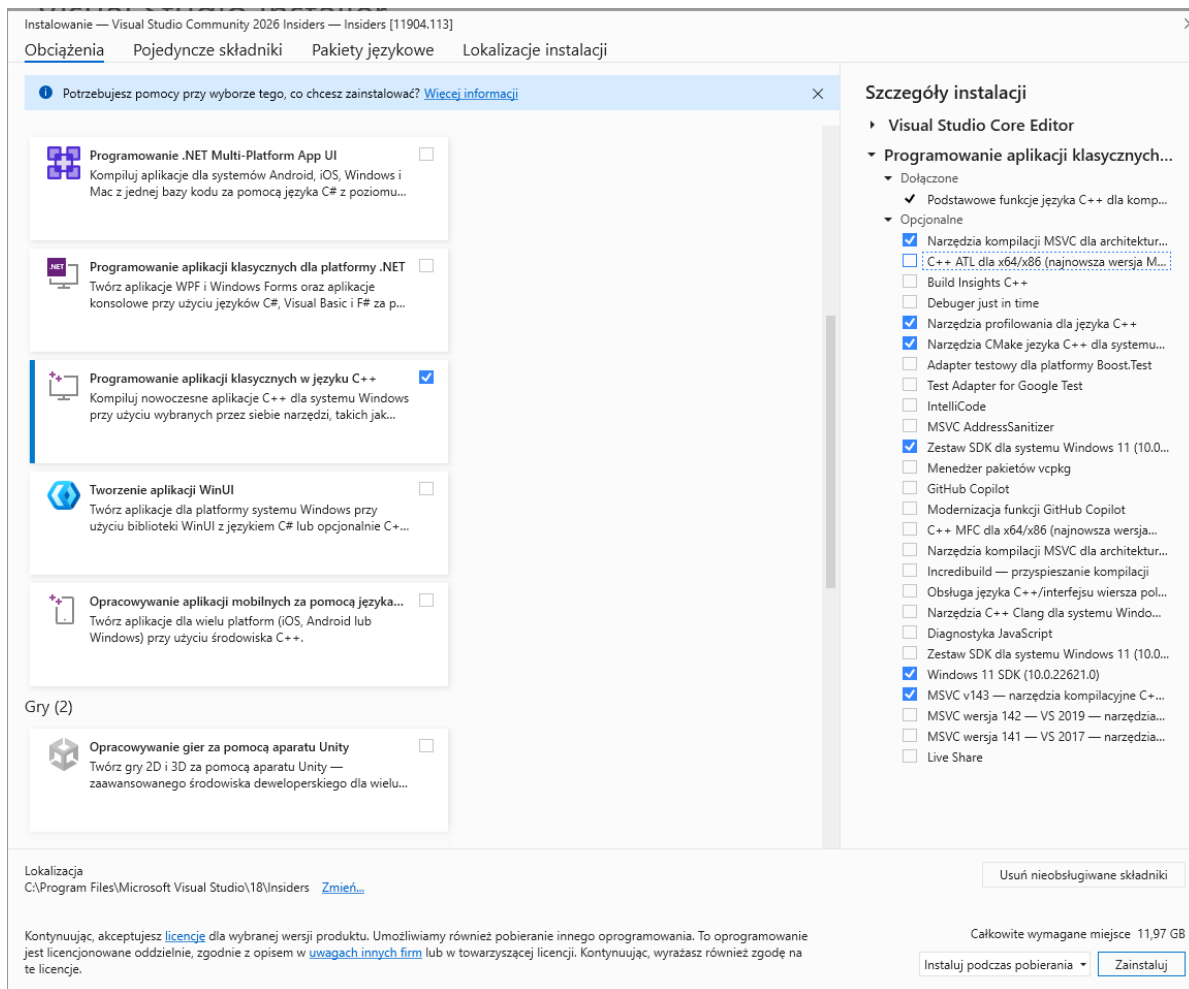
```
C:\msys64\mingw64\bin\g++.exe C:\Users\User\Downloads\RoguePlanet.cpp -o  
C:\Users\User\Desktop\roguePlanet.exe -DUNICODE -D_UNICODE
```

Po usunięciu problemów związanych z Unicode pozostały jednak błędy wskazujące na głębszą niezgodność kodu z MinGW. Dotyczyły one m.in. ponownych definicji struktur FILE_BASIC_INFORMATION oraz FILE_RENAME_INFORMATION, które były zdefiniowane zarówno w kodzie źródłowym, jak i w nagłówkach MinGW. Dodatkowo wersja FILE_RENAME_INFORMATION dostępna w MinGW różniła się od tej oczekiwanej przez kod, m.in. brakiem pola Flags.

Pojawiły się także błędy wynikające z bardziej restrykcyjnego podejścia kompilatora g++ do typów, szczególnie przy flagach enumów oraz wskaźnikach do funkcji. Dotyczyło to m.in. typów VIRTUAL_DISK_ACCESS_MASK, ATTACH_VIRTUAL_DISK_FLAG oraz przekazywania wskaźników do funkcji jako void*. W rezultacie uznano, że MinGW nie jest właściwym środowiskiem do kompilacji tego kodu bez istotnych modyfikacji źródła.

5. Przejsie na MSVC i Windows SDK

Ze względu na problemy zgodności z MinGW przygotowano środowisko MSVC oraz Windows SDK. Wybrano obciążenie „Programowanie aplikacji klasycznych w języku C++” w instalatorze Visual Studio, ponieważ analizowany kod był aplikacją natywną dla systemu Windows, napisaną w C/C++ i korzystającą bezpośrednio z Windows API oraz elementów Windows SDK. Nie był to projekt .NET, Python, Node.js ani aplikacja webowa, dlatego nie instalowano komponentów związanych z tymi technologiami.

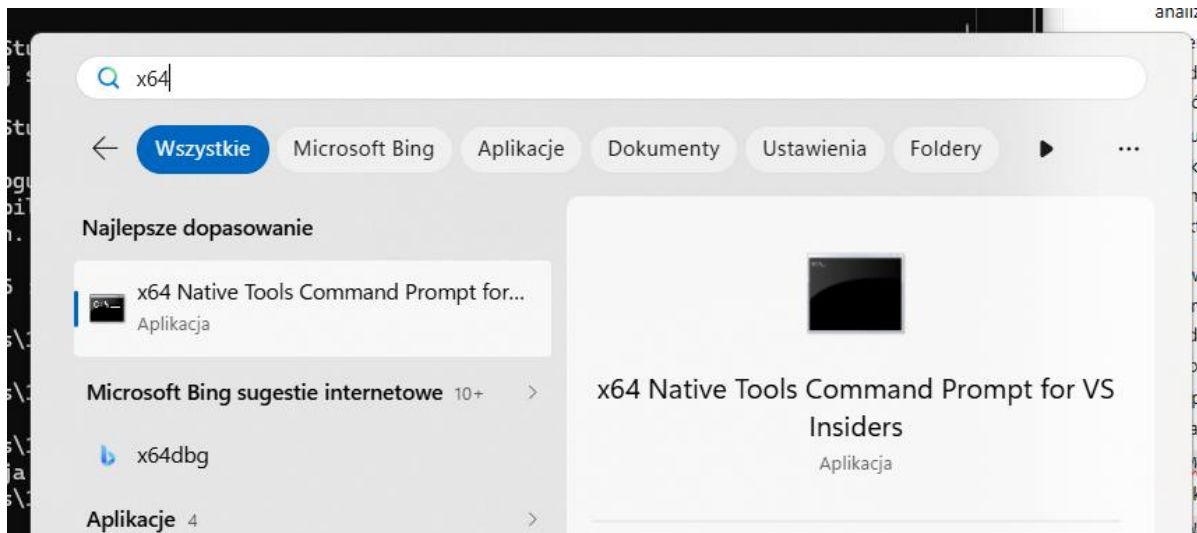


Rysunek 5. Wybrane obciążenie i składniki instalacji Visual Studio dla aplikacji klasycznych C++.

Najważniejszym składnikiem był MSVC v143, czyli kompilator Microsoft C/C++ przeznaczony do budowania aplikacji C/C++ dla systemu Windows. Wybrano go, ponieważ wcześniejsza próba kompilacji za pomocą MinGW/G++ powodowała błędy zgodności nagłówek, typów i struktur NT API. Kod korzystał z mechanizmów specyficznych dla Windows, dlatego najbardziej zgodnym środowiskiem był kompilator Microsoftu wraz z bibliotekami dostarczonymi przez Windows SDK.

Zainstalowano również Windows 11 SDK. Komponent ten zawiera nagłówki i biblioteki wymagane do korzystania z funkcji systemu Windows, m.in. windows.h, winnt.h, winreg.h, processthreadsapi.h, vtdisk.h oraz biblioteki importu .lib używane podczas linkowania. Dodatkowo pozostawiono narzędzia CMake języka C++ jako komponent pomocniczy, przydatny przy analizie bardziej złożonych projektów.

Po instalacji wykorzystano *x64 Native Tools Command Prompt for VS Insiders* – CLI z prawidłowo ustawionymi ścieżkami do kompilatora cl.exe, Windows SDK oraz bibliotek linkera.



Rysunek 6. Uruchomienie x64 Native Tools Command Prompt for VS Insiders.

6. Kompilacja za pomocą MSVC

Po przejściu na MSVC kod przeszedł znacznie dalej w procesie budowania. Pierwsza komenda nadal zwracała błędy związane z mapowaniem funkcji Windows API na warianty ANSI, dlatego konieczne było dodanie definicji UNICODE oraz `_UNICODE` także w kompilacji MSVC.

```
cl /EHsc RoguePlanet.cpp -o rogue.exe
```

```
c:\Users\User\Downloads>cl /EHsc RoguePlanet.cpp -o rogue.exe
Microsoft (R) C/C++ Optimizing Compiler Version 19.51.36247 for x64
Copyright (C) Microsoft Corporation. Wszystkie prawa zastrzeżone.
```

Rysunek 7. Próba kompilacji za pomocą MSVC bez pełnej konfiguracji Unicode i linkowania.

Po dodaniu przełączników Unicode kod został przetworzony dalej, a błędy zgodności nagłówków i typów zostały zastąpione błędami linkera LNK2019. Oznaczało to, że kompilator był już w stanie utworzyć plik obiektowy, natomiast linker nie otrzymał jeszcze wszystkich bibliotek importu wymaganych przez użyte funkcje Windows API.

```
cl /EHsc /DUNICODE /D_UNICODE RoguePlanet.cpp /Fe:rogue.exe
```

```

/out:rogue.exe
RoguePlanet.obj
RoguePlanet.obj : error LNK2019: nierozpoznany symbol zewnętrzny __imp_CreateProcessAsUserW przywołany w funkcji "void __cdecl LaunchConsoleInSessionId(unsigned long)" (?LaunchConsoleInSessionId@YAXK@Z)
RoguePlanet.obj : error LNK2019: nierozpoznany symbol zewnętrzny __imp_OpenProcessToken przywołany w funkcji "bool __cdecl IsRunningAsLocalSystem(void)" (?IsRunningAsLocalSystem@@YA_NXZ)
RoguePlanet.obj : error LNK2019: nierozpoznany symbol zewnętrzny __imp_AdjustTokenPrivileges przywołany w funkcji "int __cdecl SetPrivilege(void *,wchar_t const *,int)" (?SetPrivilege@@YAHPEAXPEB_WH@Z)
RoguePlanet.obj : error LNK2019: nierozpoznany symbol zewnętrzny __imp_DuplicateTokenEx przywołany w funkcji "void __cdecl LaunchConsoleInSessionId(unsigned long)" (?LaunchConsoleInSessionId@YAXK@Z)
RoguePlanet.obj : error LNK2019: nierozpoznany symbol zewnętrzny __imp_GetTokenInformation przywołany w funkcji "bool __cdecl IsRunningAsLocalSystem(void)" (?IsRunningAsLocalSystem@@YA_NXZ)
RoguePlanet.obj : error LNK2019: nierozpoznany symbol zewnętrzny __imp_IsWellKnownSid przywołany w funkcji "bool __cdecl IsRunningAsLocalSystem(void)" (?IsRunningAsLocalSystem@@YA_NXZ)
RoguePlanet.obj : error LNK2019: nierozpoznany symbol zewnętrzny __imp_SetTokenInformation przywołany w funkcji "void __cdecl LaunchConsoleInSessionId(unsigned long)" (?LaunchConsoleInSessionId@YAXK@Z)
RoguePlanet.obj : error LNK2019: nierozpoznany symbol zewnętrzny __imp_LookupPrivilegeValueW przywołany w funkcji "int __cdecl SetPrivilege(void *,wchar_t const *,int)" (?SetPrivilege@@YAHPEAXPEB_WH@Z)
RoguePlanet.obj : error LNK2019: nierozpoznany symbol zewnętrzny __imp_RegCloseKey przywołany w funkcji "bool __cdecl GetWDInstallDir(wchar_t *)" (?GetWDInstallDir@@YA_NPEA_W@Z)
RoguePlanet.obj : error LNK2019: nierozpoznany symbol zewnętrzny __imp_RegOpenKeyExW przywołany w funkcji "bool __cdecl GetWDInstallDir(wchar_t *)" (?GetWDInstallDir@@YA_NPEA_W@Z)
RoguePlanet.obj : error LNK2019: nierozpoznany symbol zewnętrzny __imp_RegQueryValueExW przywołany w funkcji "bool __cdecl GetWDInstallDir(wchar_t *)" (?GetWDInstallDir@@YA_NPEA_W@Z)
rogue.exe : fatal error LNK1120: liczba nierozpoznanych elementów zewnętrznych: 11

```

Rysunek 8. Błędy linkera LNK2019 dla funkcji Windows API.

Błędy linkera obejmowały funkcje takie jak `CreateProcessAsUserW`, `OpenProcessToken`, `AdjustTokenPrivileges`, `DuplicateTokenEx`, `GetTokenInformation`, `LookupPrivilegeValueW`, `RegOpenKeyExW` i `RegQueryValueExW`. Są to funkcje związane z tokenami bezpieczeństwa, uprawnieniami, uruchamianiem procesów w określonym kontekście użytkownika oraz odczytem rejestru systemowego. Sam nagłówek deklaruje istnienie funkcji, ale linker musi otrzymać właściwą bibliotekę importu wskazującą, gdzie znajdują się implementacje tych funkcji.

W celu rozwiązania błędów linkera dodano bibliotekę `advapi32.lib`. Jest to biblioteka importowa systemu Windows udostępniająca m.in. funkcje związane z tokenami bezpieczeństwa, uprawnieniami, kontami użytkowników oraz rejestrem systemowym. Po dodaniu jej do etapu linkowania linker mógł rozwiązać wcześniej nierozpoznane symbole zewnętrzne i utworzyć plik wykonywalny.

```
cl /EHsc /DUNICODE /D_UNICODE RoguePlanet.cpp /Fe:rogue.exe /link advapi32.lib
```

```

c:\Users\User\Downloads>cl /EHsc /DUNICODE /D_UNICODE RoguePlanet.cpp /Fe:rogue.exe /link advapi32.lib
Microsoft (R) C/C++ Optimizing Compiler Version 19.51.36247 for x64
Copyright (C) Microsoft Corporation. Wszystkie prawa zastrzeżone.

RoguePlanet.cpp

```

Rysunek 9. Wynik działania poprawionej komendy kompilacji.

```

/out:rogue.exe
advapi32.lib
RoguePlanet.obj

```

Rysunek 10. Wynik działania poprawnej komendy.

```

c:\Users\User\Downloads>dir rog*
Volume in drive C is Windows-SSD
Volume Serial Number is 527C-6F7A

Directory of c:\Users\User\Downloads

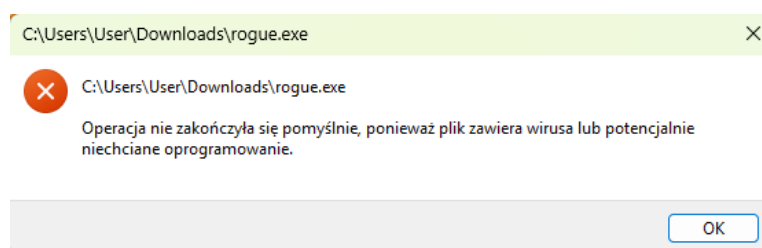
10.06.2026  22:09          1 101 312 rogue.exe
10.06.2026  20:42          5 766 226 RoguePlanet.cpp
10.06.2026  22:09          969 985 RoguePlanet.obj
               3 File(s)        7 837 523 bytes
               0 Dir(s)      198 665 048 064 bytes free

```

Rysunek 11. Utworzony plik rogue.exe w katalogu roboczym ~\Downloads\.

Reakcja Microsoft Defender i wynik uruchomienia

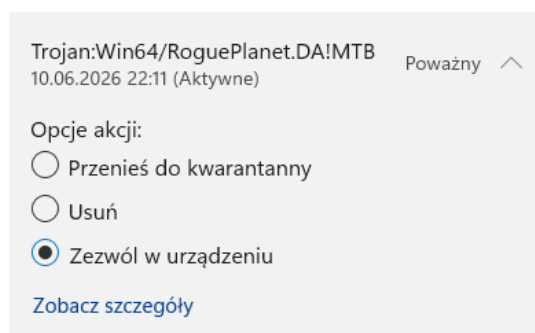
W trakcie walidacji utworzony plik wykonywalny został natychmiast wykryty przez Microsoft Defender jako Trojan:Win64/RoguePlanet.DA!MTB z poziomem zagrożenia „Poważny”. System zaproponował standardowe akcje ochronne, takie jak przeniesienie do kwarantanny lub usunięcie pliku.



Rysunek 12. Komunikat systemu Windows informujący o zablokowaniu pliku jako wirusa lub potencjalnie niechcianego oprogramowania.

Bieżące zagrożenia

Znaleziono zagrożenia. Uruchom zalecane akcje.

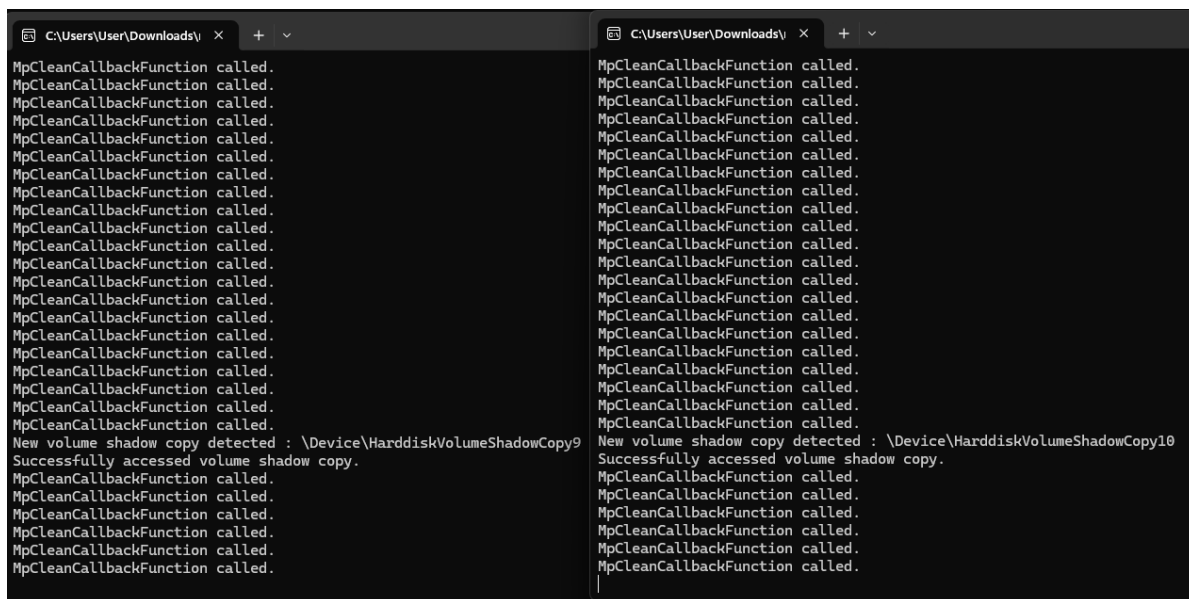


Rysunek 13. Detekcja Microsoft Defender: Trojan:Win64/RoguePlanet.DA!MTB.

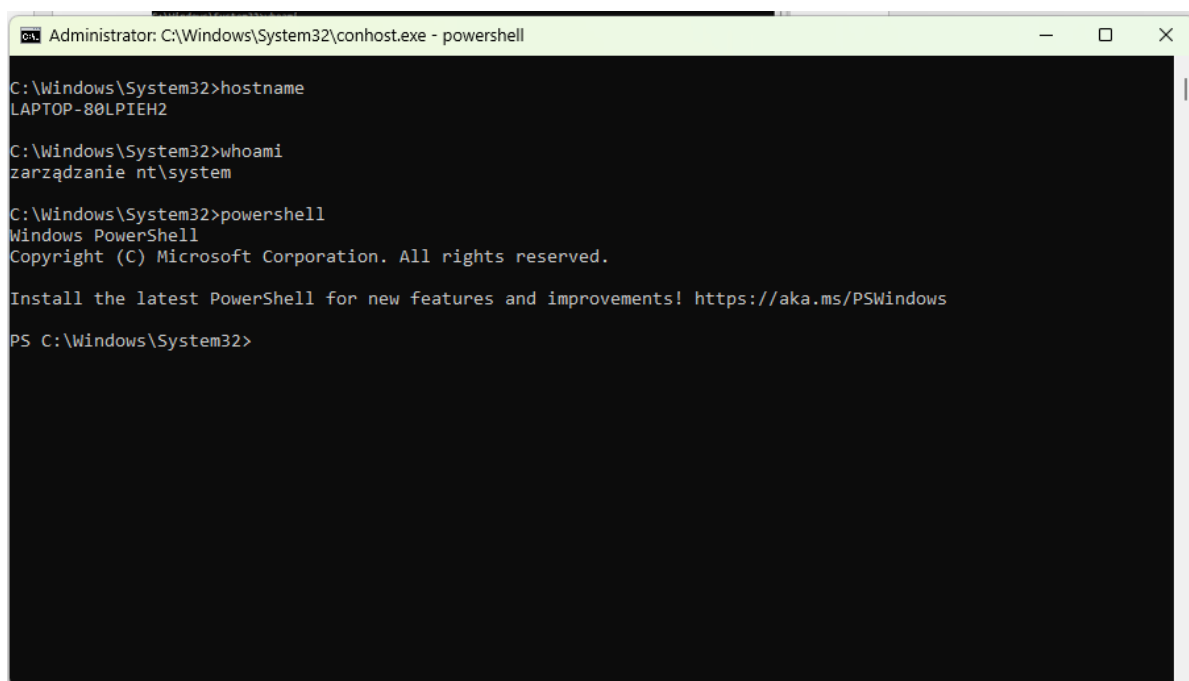
Oznacza to, że mechanizmy detekcyjne Defendera rozpoznały przygotowany artefakt jako złośliwy lub potencjalnie niebezpieczny jeszcze przed jego skutecznym uruchomieniem. Z punktu widzenia ochrony

endpointu jest to pozytywny wynik, ponieważ blokada następuje na etapie pliku wykonywalnego, a nie dopiero po obserwacji skutków działania programu.

Po czasowym wyłączeniu ochrony w czasie rzeczywistym plik można było uruchomić. Z obserwacji testowej wynika, że po drugim uruchomieniu możliwe było uzyskanie konsoli działającej z uprawnieniami SYSTEM. Wynik ten potwierdza, że aktywna ochrona Defendera miała istotne znaczenie dla zablokowania badanego artefaktu.



Rysunek 14. Uruchomienie programu w środowisku testowym po wyłączeniu ochrony w czasie rzeczywistym.



Rysunek 15. Konsola działająca w kontekście systemowym w środowisku testowym.

Ocena ryzyka

Samo wykrycie konkretnego pliku przez Microsoft Defender nie oznacza całkowitego usunięcia ryzyka podatności. Defender wykrył znany lub podobny artefakt PoC, natomiast zmodyfikowana wersja kodu, inna kompilacja, zmiana struktury pliku albo inny loader mogłyby zachowywać się inaczej względem detekcji sygnaturowej lub heurystycznej. Wynik testu należy traktować jako potwierdzenie skuteczności obecnej warstwy ochronnej wobec badanego artefaktu, a nie jako dowód, że każda możliwa wariacja techniki zostanie zablokowana.

Jednocześnie wynik testu wskazuje, że przy aktywnej ochronie w czasie rzeczywistym i aktualnych sygnaturach Defender skutecznie zablokował utworzony artefakt. Ryzyko praktycznego wykorzystania znacząco rośnie w sytuacji, gdy użytkownik ma możliwość wyłączenia ochrony w czasie rzeczywistym, dodania wyjątku, zezwolenia na wykryte zagrożenie albo lokalnej modyfikacji konfiguracji ochrony.

W praktyce oznacza to, że skuteczna mitygacja nie powinna opierać się wyłącznie na samej obecności Defendera, lecz również na centralnym wymuszaniu jego konfiguracji i blokowaniu lokalnych zmian wykonywanych przez użytkowników.

9. Zalecenia

Kluczowe znaczenie ma centralne wymuszanie konfiguracji Microsoft Defender za pomocą polityk bezpieczeństwa. Użytkownicy lokalni nie powinni mieć możliwości wyłączenia ochrony w czasie rzeczywistym, dodawania wyjątków, zezwalania na wykryte zagrożenia ani modyfikowania ustawień ochrony. W takim modelu użytkownik nie powinien być w stanie samodzielnie ominąć detekcji poprzez wybranie opcji typu „Zezwól w urządzeniu” albo czasowe wyłączenie ochrony.

- Wymusić centralnie ochronę w czasie rzeczywistym, ochronę w chmurze oraz automatyczne przesyłanie próbek;
- Zablokować użytkownikom możliwość zarządzania wyjątkami i akcjami wobec wykryć;
- Monitorować zdarzenia Defendera dotyczące wykryć, kwarantanny, prób zezwolenia na zagrożenie i zmian konfiguracji ochrony;
- Traktować wykrycie Trojan:Win64/RoguePlanet.DA!MTB jako zdarzenie bezpieczeństwa wymagające analizy;
- Rozważyć dodatkowe mechanizmy ograniczające uruchamianie nieautoryzowanych plików wykonywalnych, np. allow-listing aplikacji, WDAC lub AppLocker, zgodnie z możliwościami środowiska.

Wnioski końcowe

Test potwierdził, że przygotowanie artefaktu wymagało środowiska zgodnego z natywnym toolchainem Microsoftu. Próba kompilacji za pomocą MinGW/G++ ujawniła problemy zgodności z nagłówkami i strukturami NT API, natomiast przejście na MSVC oraz Windows SDK pozwoliło przejść do etapu linkowania i finalnie utworzyć plik wykonywalny po dołączeniu właściwej biblioteki importu.

Microsoft Defender działający w trybie normalnym, z aktualnymi sygnaturami i włączoną ochroną w czasie rzeczywistym, wykrył utworzony plik jako Trojan:Win64/RoguePlanet.DA!MTB i zablokował jego uruchomienie. Jest to pozytywny wynik testu z punktu widzenia ochrony endpointu.

Jednocześnie wyłączenie ochrony w czasie rzeczywistym umożliwiło uruchomienie artefaktu i doprowadziło do uzyskania konsoli z uprawnieniami SYSTEM. Wniosek praktyczny jest jednoznaczny: konfiguracja Defendera powinna być wymuszana centralnie, a użytkownicy nie powinni mieć możliwości lokalnego osłabiania ochrony, dodawania wyjątków ani zezwalania na wykryte zagrożenia.